

Pre-IPO Perps — Final Technical Documentation

Repository: pre-ipo-perps (dextrlabs-dev/Pre-IPO-Perpetuals-Cardano) **Document status:** Final, version-tagged for project close-off. **Scope of this document:** Testing outcomes, launch procedures, and operational guidelines for the synthetic valuation-index marketplace (Anthropic, OpenAI, SpaceX) on Cardano.

The architectural baseline lives in [MVP-TECHNICAL-DOCUMENTATION.md](#) — module breakdown, on-chain design, datum / redeemer rules, and off-chain API surface. This document is the **terminal companion**: what was tested, how the system is launched, and how it is operated post-launch.

1. Executive summary

The Pre-IPO Perpetuals MVP is feature-complete and validated end-to-end on **Cardano Preprod**:

- A single Aiken Plutus V3 spend validator (preipo) enforces oracle updates, mint, redeem, and constant-product swaps with preserved native value, monotonic seq, bounded oracle moves (`max_variance_bps = 2000`), TWAV blending (9/10), and pool deviation guards.
- A Deno HTTP API over Lucid Evolution wraps every redeemer path, serves an OpenAPI / Swagger UI surface, runs a configurable oracle worker, and persists the per-market last-spend tx for chaining.
- A Vite + React trader UI exercises markets, mint / redeem, swap, and a Trade-history view fed by the embedded Preprod evidence CSV.
- A **15-transaction Preprod batch** (3 markets × 5 redeemer steps) was executed end-to-end with all hashes recorded in `docs/preprod-run-2026-03-23T10-48-17-980Z.csv` and surfaced in the UI.

The system is **launch-ready** under the operational profile described in §3–§5: emulator for development, Preprod for the public demo, with a documented promotion path to mainnet that is out-of-scope for this funding but architecturally supported.

2. Testing outcomes

2.1 Layers exercised

Layer	Test surface	Result
Aiken validator	aiken check runs the property + unit suites in validators/preipo.ak (oracle bounds, mint/redeem accounting, swap constant-product, value preservation, seq monotonicity).	All checks pass against the committed blueprint (plutus.json).
Off-chain builder	cli/preipo-evo-core.ts Lucid-Evolution paths exercised by cli/e2e-three-markets.ts and cli/preprod-three-markets-report.ts.	Builder produces transactions accepted by both the Lucid emulator and Cardano Preprod.
HTTP API	offchain-platform-backend/main.ts + src/app.ts, validated against openapi.json. Includes /health, /api/v1/markets, lock / oracle / mint / redeem / swap, USDCx faucet, and reference-candle proxy.	All endpoints respond, idempotent on intent chaining via session-store.ts.
Frontend	preipo-frontend/src/* against the live API: balances, mint, redeem, swap, charts, and Trade-history.	UI consumes the embedded Preprod CSV and updates in-place as new tx hashes append.
End-to-end (Preprod)	cli/preprod-three-markets-report.ts walks each market through: lock → oracle bump → mint → swap → redeem , exporting a CSV row per step.	15 / 15 transactions submitted, accepted, and visible on preprod.cardanoscan.io.

2.2 Preprod evidence — recorded run

The canonical end-to-end Preprod evidence file is [docs/preprod-run-2026-03-23T10-48-17-980Z.csv](#) (also embedded in preipo-frontend/src/data/preprodLedger.ts as PREPROD_LEDGER_CSV). One row per redeemer step, with explorer URLs and a datum snapshot per tx.

Market	Step	Tx hash	Explorer
anthropic	lock	b15cd6d048304c9ade05fe36bb82c5d6279da9de0d4b66b6ffadd701d0657b34	view
anthropic	oracle (+1%)	a4e80d92dd3cbcd228e996bd027d122f8a5bf7bc6cff52d976958d6236073cb2	view
anthropic	mint (qty=5000)	2c6034e8cdcab97d2bace4d439c43598039bd12068d287de983d391f8d4894ae	view
anthropic	swap (sellA, in=8000)	0389c0da166602894b09bca762ca9fc20df9aece0124a28ab7c9712ae23be4df	view

Market	Step	Tx hash	Explorer
anthropic	redeem (burn=2000)	7743d8340c107206afd568e341318615c1ab4740c45e055558e7f460f5707b4e	view
openai	lock	5e19c84ee7dd313a7f2c92a9fbf55b7b19eb0e3d0460da22f8acd1460c851692	view
openai	oracle (+1%)	f81c7037eb5945a645147094f2fccb0c5285044a2fab7292aea5fb953311418	view
openai	mint (qty=5000)	e297b02842166871f03507b41585ddb4ed50f43f38429b8383beb4ce4f9adfae	view
openai	swap (sellA, in=8000)	2091531c02a5bcd3fb832f2bc9fe59774c9904369de0bdba4f24b416770c57f1	view
openai	redeem (burn=2000)	adf0696d88c19146a8cc01c6f7a6aa773f356fea5c44d7d94ad940a738332933	view
spacex	lock	9c0f3fcec7edb0cc0e5458b790015941a1895fce7abb8b73138b8abcbcd39aaa	view
spacex	oracle (+1%)	544148d49c7fa3dd9c75a7e3da5608a9274de9461e6399c0ed396665f3a91f0a	view
spacex	mint (qty=5000)	7d274822e6092555f772bf2ae4da6f13d669c6d20598b16b5d7ca64eafb1b088	view
spacex	swap (sellA, in=8000)	9a884a1b9dc350b5c8aaf30a54c4c4b8d29cf8c36670231c000e249c293e8e58	view
spacex	redeem (burn=2000)	5ac137ac0ea90bf6c05a4c7ad111be1ef9c6506bd5dce2eb31276afad51f5b73	view

2.3 Quantitative outcomes

Metric	Result
Preprod transaction success rate	15 / 15 (100%)
Markets exercised	Anthropic, OpenAI, SpaceX (3 / 3 planned)
Redeemer paths exercised	Oracle, Mint, Redeem, Swap — 4 / 4
Validator checks	aiken check passes; blueprint regenerable via aiken build
API contract	openapi.json served at /openapi.json; Swagger at /docs

2.4 Known issues + fixes applied during testing

Symptom	Root cause	Resolution
Swap UI quoting in micro units confused traders	Frontend converted between μ -scale and display inconsistently when the sellA toggle flipped	fix(frontend): swap amounts and quote in USD / whole tokens — swap form normalizes to whole tokens before submit (commit 21b4393)
Mint button enabled while wallet had insufficient USDCx	Faucet ledger compared raw integer to μ -scale value	fix(frontend): mint button USDCx check uses token scale (commit 1c25fe9)
Preprod tx chaining occasionally referenced a stale UTxO after redeploy	session-store.ts retained the previous lock hash across emulator → preprod switches	Added explicit marketId keying + a reset flag on backend boot when PREIPO_USE_PREPROD=1
Oracle worker over-submitted when the reference feed paused	Worker compared timestamps to wall-clock, not last on-chain last_oracle_time	Worker now uses the on-chain last_oracle_time + min_interval_ms from datum before submitting

3. Launch procedures

3.1 Pre-launch checklist

1. **Build the validator** — aiken build (regenerates plutus.json).
2. **Run validator checks** — aiken check.
3. **Backend dependencies** — cd offchain-platform-backend && deno cache main.ts.
4. **Frontend dependencies** — cd preipo-frontend && npm install.
5. **Smoke against emulator** — deno task serve:emulator + npm run dev:emulator; confirm GET /health returns ok and the UI loads with three markets.
6. **Preprod environment** — populate .env with BLOCKFROST_PROJECT_ID, PREIPO_WALLET_SEED, PREIPO_USE_PREPROD=1. **Do not commit secrets.**
7. **Preprod evidence re-run (optional)** — deno run -A cli/preprod-three-markets-report.ts — verifies the full end-to-end cycle against a funded Preprod wallet, writes a fresh docs/preprod-run-*.csv, and the resulting CSV can be embedded into preipo-frontend/src/data/preprodLedger.ts to refresh Trade history.

3.2 Launch sequence (Preprod public demo)

Step 1	Build validator	aiken build
Step 2	Start backend (Preprod mode)	deno task serve:preprod
Step 3	Start frontend	npm run dev:preprod
Step 4	Verify openapi.json + /docs	curl :PORT/openapi.json
Step 5	Confirm three markets loaded	GET /api/v1/markets
Step 6	Oracle worker on	backend env PREIPO_ORACLE_WORKER=1
Step 7	Public-facing reverse proxy	Cloudflare Tunnel (or equivalent)

3.3 Environment variables (operational)

Variable	Where	Purpose
PREIPO_USE_PREPROD	Backend, CLI	Selects Blockfrost-backed Lucid provider for Preprod
PREIPO_USE_EMULATOR	Backend	Selects in-process Lucid emulator (dev)
BLOCKFROST_PROJECT_ID, BLOCKFROST_URL	Backend, CLI	Preprod Cardano provider
PREIPO_WALLET_SEED	Backend, CLI	24-word seed for the operator wallet (never committed)
PREIPO_ORACLE_WORKER	Backend	Toggles the in-process oracle worker
PREIPO_ORACLE_INTERVAL_MS	Backend	Lower bound between oracle submissions
PREIPO_FAUCET_DIR	Backend	Override for the USDCx faucet JSON store
PREIPO_SESSION_DIR	Backend	Override for last-spend tx persistence

A full table lives in [SETUP.md](#).

3.4 Rollback / recovery

- The validator address is deterministic from `plutus.json`. To roll back a frontend / backend release without touching on-chain state, redeploy the previous tag — the validator continues to accept transitions from the latest seq.
- Each market is independent at the script level. If a single market's last-spend tx is lost from the session store, recover it by querying Blockfrost for the latest UTxO at the validator address whose datum's `index_id` matches the market — `cli/preprod-three-markets-report.ts` performs exactly this lookup on each step.
- The USDCx faucet ledger is a JSON file (`offchain-platform-backend/data/`). Take a snapshot before each release; restore by file replacement if a faulty release writes inconsistent balances.

4. Operational guidelines

4.1 Day-2 ownership matrix

Component	Owner	Cadence
Aiken validator + plutus.json	Smart-contract engineer	On-change only (no auto-deploy)
Off-chain backend (offchain-platform-backend)	Backend engineer	Continuous on main
Frontend (preipo-frontend)	Frontend engineer	Continuous on main
Oracle worker config (PREIPO_ORACLE_INTERVAL_MS, max_variance_bps)	Operations	Reviewed quarterly
USDCx faucet ledger	Operations	Snapshot per release
Preprod evidence CSV refresh	QA	After any redeemer-affecting change

4.2 Service-level objectives (Preprod public demo)

SLO	Target	Source of truth
API availability	99% monthly	Backend healthcheck (GET /health)
Oracle freshness	$\leq \text{min_interval_ms} \times 2$	On-chain last_oracle_time vs wall-clock
Frontend load	≤ 3 s cold	Browser devtools / synthetic ping
Preprod tx success rate	$\geq 95\%$	cli/preprod-three-markets-report.ts

4.3 Change-management

- All on-chain changes go through aiken check + a fresh Preprod evidence re-run before being merged.
- Off-chain breaking changes are gated by an openapi.json diff in PR description.
- Frontend changes that alter Preprod tx semantics must update preipo-frontend/src/data/preprodLedger.ts in the same PR.

5. Reproducibility

Anyone can reproduce the close-out state by:

```
git clone https://github.com/dextrlabs-dev/Pre-IP0-Perpetuals-Cardano.git
cd Pre-IP0-Perpetuals-Cardano
aiken build && aiken check
cd offchain-platform-backend && deno task serve:emulator # in terminal A
cd ../preipo-frontent && npm install && npm run dev:emulator # in terminal B
```

For the Preprod path:

```
export PREIPO_USE_PREPROD=1
export BLOCKFROST_PROJECT_ID=...
export PREIPO_WALLET_SEED='...'
deno run -A cli/preprod-three-markets-report.ts # writes docs/preprod-run-*.csv
```

The resulting CSV is byte-comparable (modulo wallet UTxOs + timestamps) to the committed docs/preprod-run-2026-03-23T10-48-17-980Z.csv.

6. Document control

Item	Value
Version	final-v1.0
Companion documents	MVP-TECHNICAL-DOCUMENTATION.md , POST-LAUNCH-OPERATIONS.md , CLOSE-OUT-REPORT.md
Evidence	preprod-run-2026-03-23T10-48-17-980Z.csv
Render	deno run -A docs/render-pdf.ts (point mdPath at this file for the PDF export)
License	Inherits repository license